

KẾ HOẠCH PHÂN CHIA CÔNG VIỆC NHÓM 5 NGƯỜI

Dự án: Hệ thống trực quan hóa cấu trúc rời rạc & giải thuật đồ thị

I. BẢNG PHÂN CHIA CÔNG VIỆC (Mỗi thành viên làm 1 nhóm thuật toán)

Đảm bảo tất cả 5 thành viên đều phát triển cả hai phần: Cài đặt thuật toán thuần (Backend) và Trực quan hóa hoạt họa trên giao diện (Frontend/GUI).

Thành viên	Nhóm Thuật toán chịu trách nhiệm	Chi tiết phần việc (Thuật toán + GUI)
Thành viên 1	Duyệt đồ thị & Biểu diễn đồ thị	- Viết mô-đun chuyển đổi cấu trúc (Ma trận kề, ds kề, ds cạnh cho cả đồ thị vô hướng & có hướng) ở utils/conversions.py. - Cài đặt thuật toán duyệt BFS & DFS thuần và generator. - Tích hợp hiển thị biểu diễn đồ thị và hoạt họa BFS/DFS lên GUI.
Thành viên 2	Đường đi ngắn nhất & Đồ thị hai phía	- Cài đặt giải thuật Dijkstra sử dụng Priority Queue (Min-Heap), kiểm soát trọng số âm. - Cài đặt thuật toán kiểm tra Đồ thị 2 phía (tô màu BFS/DFS), truy vết ngược lấy chu trình lẻ làm minh chứng phản ví dụ. - Tích hợp highlight lộ trình Dijkstra và vẽ màu đỏ/xanh (hoặc highlight chu trình lẻ) lên GUI.
Thành viên 3	Cây khung nhỏ nhất (MST)	- Cài đặt thuật toán Prim (dùng Heap). - Cài đặt thuật toán Kruskal sắp xếp cạnh kết hợp cấu trúc dữ liệu tập hợp rời rạc (DSU) để phát hiện chu trình nhanh $O(1)$. - Tích hợp trực quan hóa cây khung nhỏ nhất (MST) và tính tổng trọng số lên GUI.
Thành viên 4	Luồng cực đại (Max Flow)	- Cài đặt thuật toán Edmonds-Karp (phương pháp Ford-Fulkerson tối ưu hóa đường đi ngắn nhất bằng BFS trên mạng dư). - Tích hợp hiển thị tỷ lệ flow/capacity thực tế trên các cạnh và highlight các đường tăng luồng qua từng bước. - Phụ giúp Thành viên 1 xây dựng khung Canvas vẽ đồ thị chính.
Thành viên 5	Đường đi & Chu trình Euler	- Cài đặt giải thuật **Fleury** (duyet kiểm tra và tránh các cạnh cầu - bridge). - Cài đặt giải thuật **Hierholzer** tối ưu $O(E)$ sử dụng cơ chế lưu trữ Stack. - Tích hợp mô phỏng bước đi của Fleury và cơ chế push/pop đỉnh vào Stack của Hierholzer lên GUI.

II. HƯỚNG DẪN TRIỂN KHAI THEO THỨ TỰ (4 Giai đoạn)

Giai đoạn 1: Xây dựng nền tảng và Thuật toán thuần (Tuần 1 - 2)

- Thứ tự thực hiện: Thành viên 1 thiết lập các cấu trúc dữ liệu đồ thị chung và hàm chuyển đổi trong `utils/conversions.py`. Thành viên 4 cùng lúc thiết lập khung giao diện cơ bản (cửa sổ chính, các nút chức năng, bảng kết quả log).
- Triển khai thuật toán: Tất cả các thành viên viết giải thuật thuần túy của mình bên trong thư mục `algorithms/pure/`. Các thuật toán này chỉ nhận tham số đầu vào và trả về kết quả cuối cùng, kiểm thử trực tiếp bằng CLI.

Giai đoạn 2: Phát triển các bộ sinh Hoạt họa (Tuần 3)

- Thứ tự thực hiện: Chuyển đổi thuật toán từ dạng hàm thông thường sang dạng **Generator** (sử dụng `yield`) trong thư mục `algorithms/`.
- Mục tiêu: Ở mỗi bước lặp chính (ví dụ: duyệt qua một cạnh, tô màu một đỉnh, tìm thấy một đường đi), thuật toán sẽ `yield` ra trạng thái hiện tại (danh sách đỉnh đã thăm, tập cạnh tích lũy, các biến trung gian).

Giai đoạn 3: Thiết kế Giao diện Canvas và Liên kết hoạt họa (Tuần 4)

- Thứ tự thực hiện: Thành viên 1 hoàn thiện Canvas vẽ đồ thị. Từng thành viên thực hiện viết hàm điều khiển vẽ trạng thái của thuật toán mình trên Canvas (dùng các màu khác nhau để biểu thị trạng thái đỉnh/cạnh) và in log giải thích tương ứng.
- Mục tiêu: Khi chọn thuật toán và click 'Chạy hoạt họa', GUI sẽ liên tục lấy trạng thái tiếp theo từ Generator và vẽ lại đồ thị sau mỗi khoảng thời gian delay nhất định.

Giai đoạn 4: Tích hợp Đồ thị mẫu, Thử nghiệm & Kiểm thử (Tuần 5)

- Thứ tự thực hiện: Cài đặt chức năng nạp đồ thị mẫu cho mỗi thuật toán (mỗi thành viên tự vẽ một đồ thị mẫu chuẩn mực nhất để nạp mặc định cho giải thuật của mình, giúp người dùng dễ dàng thử nghiệm nhanh mà không cần vẽ tay).
- Mục tiêu: Thực hiện test chéo (cross-testing) giữa các thành viên, sửa các lỗi giao diện và tối ưu hóa code.

III. YÊU CẦU ĐẠT ĐƯỢC (Định nghĩa Hoàn thành - Definition of Done)

1. Yêu cầu về thuật toán (Backend)

- Đúng đắn và tối ưu: Thuật toán phải giải quyết đúng bài toán, đạt độ phức tạp thời gian tiêu chuẩn (Dijkstra/Prim đạt $O(E \log V)$, Kruskal $O(E \log E)$, Hierholzer $O(E)$).
- Tính nhất quán (Deterministic): Các đỉnh kề phải được sắp xếp theo số thứ tự tăng dần trước khi duyệt để kết quả chạy vết bằng tay và bằng máy trùng khớp hoàn toàn.
- Bẫy lỗi đầy đủ: Kiểm tra và ném ra các ngoại lệ thích hợp (ví dụ: Dijkstra có trọng số âm, Kruskal cho đồ thị có hướng, Max Flow không chọn đỉnh đích...).

2. Yêu cầu về trực quan hóa (Frontend)

- Hoạt họa mượt mà: Chạy hoạt họa từng bước không được gây đơ/treo giao diện (phải dùng giải pháp Generator lấy dữ liệu tuần tự kết hợp hàm `after()` của Tkinter).
- Màu sắc trực quan: Sử dụng màu sắc tương phản rõ ràng để phân biệt trạng thái (ví dụ: cam cho đỉnh được chọn, xanh lá cho đỉnh/cạnh đang xét, đỏ cho kết quả đường đi/MST cuối cùng).
- Nhãn thông tin rõ ràng: In chi tiết log từng bước (bước hiện tại đang xét đỉnh nào, cạnh nào được chọn, tại sao bỏ qua...) để phục vụ mục đích tự học lý thuyết.

3. Yêu cầu về mã nguồn và tài liệu

- Mã nguồn sạch (Clean Code): Đặt tên biến, hàm theo đúng chuẩn toán học. Viết docstring giải thích tham số đầu vào và đầu ra cho tất cả các hàm thuật toán.

- Tích hợp Git liên tục: Các thành viên làm việc trên các nhánh (branch) riêng biệt và merge vào nhánh main thông qua Pull Request sau khi đã kiểm thử kỹ lưỡng.